



TIYA

Accounts, sessions and security

You answer every question about how someone gets in, how they stay in, and what stops the wrong person getting further.

1. Your role in one sentence

You own the front door: creating an account, proving who a person is, keeping them signed in safely, and refusing anyone who is not allowed on a page.

2. What you own

FILE (UNDER <code>SRC/MAIN</code> UNLESS STATED)	LINES	WHAT IT IS
<code>web/LoginServlet.java</code>	181	Sign in form and the sign in decision, session id change, cookies
<code>web/RegisterServlet.java</code>	158	Student sign up: every field checked on the server
<code>web/LogoutServlet.java</code>	48	Ends the session
<code>web/SessionUser.java</code>	69	The small object kept in the session: id, email, name, role. Never the hash
<code>web/SessionTracker.java</code>	43	HttpSessionListener that counts open sessions
<code>web/ThemeServlet.java</code>	46	Light or dark palette kept in a cookie
<code>web/Web.java</code>	107	Shared helpers: forward, redirect, flash messages, CSRF token
<code>security/AuthFilter.java</code>	115	The gate in front of <code>/app/*</code> and <code>/admin/*</code> : login, role, CSRF
<code>security/RequestPaths.java</code>	36	Turns the raw request URL into the path the container routes on, and says whether it is an admin path
<code>security/PasswordHasher.java</code>	97	PBKDF2 hashing and checking, JDK only
<code>security/SecurityHeadersFilter.java</code>	88	CSP and the other protective headers
<code>ejb/AuthService.java</code>	189	The only class that compares a password. Lockout, registration, change password

<code>ejb/AuthResult.java</code>	82	The outcome of a sign in and the wording of each message
<code>util/EmailDomainVerifier.java</code>	262	Disposable domain list and the DNS check on the email domain
<code>webapp/WEB-INF/web.xml</code>	73	Session timeout, cookie flags, error pages, scriptlets off
<code>webapp/WEB-INF/views/login.jsp</code> , <code>register.jsp</code>	138 + 167	The two forms
<code>src/test/.../PasswordHasherTest.java</code> , <code>EmailDomainVerifierTest.java</code> , <code>RequestPathsTest.java</code>	83 + 76 + 49	Tests for hashing, for the email domain checks, and for the admin path rule (5 tests, part of the 146 test suite)

You also lean on `util/Validators.java` (118 lines, shared with Chandra) for the email shape, name and password strength rules.

3. The 60 second explanation

SAY THIS

A student creates an account on the register page. The server checks every field itself, and for the email it does three things: the shape must be valid, the domain must not be a known disposable inbox, and the domain must exist in DNS and be able to receive mail. We do not send a confirmation link, so we prove the domain is real, not that the mailbox is.

The password is never stored. We store a PBKDF2 hash with 120,000 rounds and a fresh random salt for each person, so two people with the same password get different hashes. On sign in we hash what was typed and compare. A wrong password and an unknown email give the same message. Five wrong attempts lock the account for fifteen minutes.

After a good sign in we change the session id, which defeats session fixation, and keep a small user object in the session. Every page under `/app` and `/admin` goes through one filter that checks there is a user, that admin pages need the officer role, and that every POST carries a CSRF token. Every response carries a Content Security Policy, and the JSP files sit under `WEB-INF` so nobody can open them by URL.

4. Code walk in VS Code

Line numbers are from the current files. Open each file, click the line, say the sentence. Do not scroll around.

- 1 `views/register.jsp` , lines 57 to 58, 83, 92. Point at `method="post"` , the hidden `csrf` input, and the two `type="password"` inputs. There is no `value` attribute on either.
Say: "Passwords are typed into masked boxes, sent by POST so they never appear in the URL, and the page is never given a password to print back, so a failed form comes back empty."

- 2** **RegisterServlet.doPost** , lines 64 to 79. Point at the checks in order: CSRF, name, then the email chain (`isEmail` , `isDisposableDomain` , `hasValidMxOrA` , `emailTaken`), then password strength.
Say: "The browser attributes are only a convenience. The server repeats every rule, and the email has four checks that run in this order."
- 3** **Validators.isEmail** , lines 20 to 23 and 34 to 43.
Say: "This is shape only: local part, at, dotted domain, last label two letters or more. `admin@campus` fails here. A made up domain passes here, which is why the next class exists."
- 4** **EmailDomainVerifier.hasValidMxOrA** , lines 140 to 159. Then `lookupViaJndi` (171) and `lookupViaHostname` (235).
Say: "It asks DNS for an MX record, then A and AAAA as RFC 5321 allows. If the lookup cannot be completed at all, for example no network, it lets the sign up through and logs a warning, so a DNS outage cannot lock every student out. If DNS answers that the domain has nothing, it is refused."
- 5** **EmailDomainVerifier.DISPOSABLE_DOMAINS** , lines 77 to 91.
Say: "Forty three well known throwaway providers, such as mailinator. They resolve fine in DNS, so they need their own list."
- 6** **AuthService.registerStudent** , lines 125 to 154. Point at line 129 (email taken again), 132 (roll number clash), 140 to 141 (`PasswordHasher.hash` and `Role.STUDENT`).
Say: "It checks duplicates a second time because two browsers can submit at once. The role is fixed to STUDENT in code. Sign up can never create an officer."
- 7** **PasswordHasher** , lines 22 to 27, 32 to 39, 45 to 63. Point at `PBKDF2WithHmacSHA256` , 120,000 iterations, 16 byte `SecureRandom` salt, the stored format, and `MessageDigest.isEqual` on line 59.
Say: "Stored as `pbkdf2$120000$salt$hash` . The salt is new for every account. The comparison takes constant time. It is JDK only, no library."
- 8** **LoginServlet.doPost** , lines 69 to 121. Read down: CSRF (81), `authService.authenticate` (87), failure path (89 to 93), `changeSessionId` (98), `SessionUser` into the session (103 to 104), cookies (108 to 112), redirect (116 to 120).
Say: "The servlet only decides where to send the person. All password logic is in one bean."
- 9** **AuthService.authenticate** , lines 46 to 92. Point at 52 (bad email shape never reaches the database), 58 to 62 (unknown email still burns a hash), 69 to 72 (locked check), 74 to 85 (the counter), 37 to 38 (the two constants).
Say: "Each wrong password adds one to `failed_attempts` in the database. At five, `locked_until` is set fifteen minutes ahead and the counter resets. While locked, even the right password is refused."
- 10** **AuthResult.badCredentials** , lines 44 to 50.
Say: "Wrong password and unknown email produce the identical sentence, so the form cannot be used to find out who has an account. Once an account has two attempts left, the message adds a countdown."

11 **RequestPaths** , lines 23 to 35, then **AuthFilter** line 53.

Say: "The container routes `//admin/database` to the same servlet as `/admin/database` , but the raw URL still has both slashes. Our first version tested that raw text, so a student could slip past the officer check with a doubled slash. Now `canonical` strips the context path and any semicolon parameter and collapses repeated slashes, and `isAdminPath` tests the cleaned path. Five tests pin it."

12 **AuthFilter** , lines 36, 58 to 61, 63 to 68, 70 to 89.

Say: "One filter guards `/app/*` and `/admin/*` . No user in the session means a redirect to login with the original URL remembered. A student on an admin path gets a 403 and the `error-403.jsp` page. A POST without the right CSRF token is bounced. If reading the token fails because an upload was over the size limit, the user gets a flash message and a redirect, not a 500 page."

13 **Web.csrfToken** and **csrfValid** , lines 73 to 101.

Say: "A 32 byte random token lives in the session and is echoed in a hidden field. A form posted from another site cannot read it."

14 **web.xml** , lines 22 to 31, and **SessionTracker** , lines 22 to 34.

Say: "Session timeout is 30 minutes, the session cookie is HttpOnly, and only cookies are allowed, no session id in the URL. The listener counts sessions created and destroyed, and the officer's System page shows the numbers."

15 **LoginServlet.writeCookie** , lines 171 to 180, and **ThemeServlet** , lines 33 to 37.

Say: "Two cookies on sign in: the email if the box was ticked, and the time of the last visit. Both HttpOnly, 30 days, never a password. The theme cookie is set by the theme servlet for a year and is deliberately readable by scripts."

16 **SecurityHeadersFilter** , lines 51 to 61, 70 to 74, 79 to 84.

Say: "Scripts may only come from our own origin, no inline script at all. Framing is refused, forms may only post to us, and anything that is not a static asset is marked no-store."

17 **Web.render** and **Web.redirect** , lines 44 to 60, then **LogoutServlet** , lines 26 to 38.

Say: "Views are forwarded from `WEB-INF/views` , so a browser cannot ask for them. After a successful POST we redirect. Logout closes the shortlist bean, calls `session.invalidate()` , and redirects to login."

5. Live demo plan: create an account and sign in

About five minutes. Use a private browser window so no old cookies interfere. The app must be running at `http://localhost:8080/placement/` . This is the same demo account the whole presentation uses: Karan hands you the register page already open, and Kinjal, Chandra and Yuvraj continue with this same account after you. Use the exact values below, not made up ones, so it matches the rest of the run sheet: name **Demo Student** , email `demo.student.cs@gmail.com` , password `Demo@2026x` , roll `CS26100` , branch Computer Science, batch year `2026` , CGPA `8.0` , backlogs `0` , phone `9876543210` .

- 1 **Open** `http://localhost:8080/placement/register` . Type anything in a password box and point at the dots. Right click, Inspect, show `type="password"` and `method="post"` .
Say: "The password is masked, and it goes in the request body, not the address bar."

- 2 **Fake email first.** Fill the form: name `Demo Student` , email `demo.student@thisdomaindoesnotexist123.com` , password `Demo@2026x` in both boxes, roll `CS26100` , branch Computer Science, batch year `2026` , CGPA `8.0` , backlogs `0` , phone `9876543210` . Click the create button.
Expect: the form comes back with "That domain doesn't appear to accept email. Check for a typo and try again." beside the email. The two password boxes are empty and the other fields are kept. In the devtools Network tab the response is `400` .
Say: "The address is well formed, so the shape check passes. The domain has no DNS record, so the second check refuses it."
Optional: use `x@mailinator.com` to show "Please use a permanent email address, not a disposable or temporary inbox."

- 3 **Valid sign up.** Change only the email to `demo.student.cs@gmail.com` and type the passwords again. Submit.
Expect: a redirect to the sign in page showing a green panel with a tick, "Welcome, Demo. Your account is ready.", the email and roll number, and three steps with step 1 highlighted. The email is already filled in, the cursor is in the password field, and the button reads "Sign in to your new account". Reload the page and the panel is gone: it is shown once (RegisterServlet puts three values in the session, LoginServlet reads and removes them).
Say: "That was a POST followed by a redirect, so pressing refresh cannot create the account twice."

- 4 **Show the stored password.** Open PowerShell and run:

```
$env:MYSQL_PWD = 'Placement@2026'
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u placement_user placement_cell -e "SELECT id, email, user_role, password_hash FROM app_user ORDER BY id DESC LIMIT 3;"
```

Expect: your new row on top with `STUDENT` and a value like `pbkdf2$120000$CX25cv2wd8tNj4Vcb1VCjg==$qBJs55Jt1cqYpZ//gtvCJZPH8ZE+IMaqZmlbD955kYA=` . Point at the four parts: algorithm, 120000 rounds, salt, hash. Password `Demo@2026x` appears nowhere.
Then show the salt matters: add `WHERE email IN ('aarti.deshpande@campus.edu', 'rahul.menon@campus.edu')` . Both seeded accounts share the password `Campus@2026` , yet the salt and hash differ.
Say: "Even we cannot read the password back. A person with the same password gets a different hash because the salt is random per account."

- 5 **Wrong password.** On the sign in page enter `demo.student.cs@gmail.com` and `Wrong@123` .
Expect: "Email or password is incorrect." with HTTP `401` , the email box kept, the password box empty. Repeat with `nobody.here@gmail.com` : the identical sentence.
Say: "We never say which half was wrong. A stranger cannot use this form to find out who has an account."

6 Correct login. Enter `demo.student.cs@gmail.com` and `Demo@2026x`, tick "Remember my email on this device", sign in.

Expect: a redirect to `/placement/app/dashboard` with "Welcome to the placement cell" on the first sign in from a browser, and "Welcome back" after that. In devtools, Application, Cookies:

`JSESSIONID`, `remember_email` and `last_visit`, all HttpOnly, path `/placement`. In Network, open any page and show the `Content-Security-Policy` response header.

Say: "The session id was replaced at this moment. The user object in the session holds the id, email and role, never the hash."

7 Student blocked from admin. Signed in as this student, type

`http://localhost:8080/placement/admin/database` in the address bar.

Expect: the "Not allowed" page, HTTP `403`. Then open a second private window (no session) and visit the same URL: it redirects to the sign in page with "Please sign in to continue." and a `next=` parameter.

Say: "Two different answers: not signed in is a redirect, signed in with the wrong role is a 403."

8 Try to get around it. Still signed in as the student, type

`http://localhost:8080/placement//admin/database` (two slashes after `placement`).

Expect: the same "Not allowed" page, HTTP `403`. `//admin//students` and `/admin;x=1/students` give 403 too.

Say: "We found this one in our own testing. The doubled slash used to get through because the check read the raw URL. Now the check reads the same cleaned path the container routes on."

9 Logout. Click the sign out icon at the top right (its tooltip says "Sign out").

Expect: the sign in page with "You have been signed out." Press the browser Back button: you land on the sign in page, not the dashboard.

Say: "The session is invalidated on the server and every page is sent with no-store, so Back cannot show a cached dashboard."

10 Optional, logout. Sign in to the new account with a wrong password five times. On the third and fourth you see "2 attempts left" and "1 attempt left before this account is locked". The fifth gives "Too many failed attempts. This account is locked for another 15 minutes." A correct password is now refused as well. Undo it: sign in as `tpo@campus.edu`, open Students, open the account, click "Clear the logout".

Never do this on `aarti.deshpande@campus.edu` or the officer account. Use your own demo account.

CAREFUL

After the demo, remove your test account: as the officer open Students, open the account, click "Remove from the register". Do not run any delete query in mysql. In the `locked_until` column the driver stores UTC, so the time can look about five and a half hours behind the clock. That is display only.

6. Servlet course topics mapped to this code

TOPIC	WHERE IN THE CODE
Servlet lifecycle	<code>LoginServlet</code> : the container creates one instance, calls <code>init()</code> once, calls <code>service()</code> for every request (which hands GET to <code>doGet</code> , line 49, and POST to <code>doPost</code> , line 69), and calls <code>destroy()</code> when the app is undeployed. One instance serves every user at once, so the class keeps no per user fields, only injected beans. The same three steps are visible in <code>AuthFilter</code> : <code>init</code> line 42, <code>doFilter</code> line 47, <code>destroy</code> line 112.
Servlet declaration	<code>@WebServlet</code> on <code>LoginServlet</code> line 33. <code>web.xml</code> holds only what has no annotation: session, error pages, JSP settings.
Request parameters, encoding	<code>request.getParameter("email")</code> line 74, <code>setCharacterEncoding("UTF-8")</code> line 72
RequestDispatcher forward	<code>LoginServlet</code> lines 65 and 132, <code>RegisterServlet</code> line 117, <code>AuthFilter</code> line 66, <code>Web.render</code> line 48
Redirect, Post-Redirect-Get	<code>Web.redirect</code> lines 57 to 60, used at <code>LoginServlet</code> line 120 and <code>RegisterServlet</code> line 126. <code>LogoutServlet</code> line 38 and <code>AuthFilter</code> lines 78, 86 and 108 call <code>sendRedirect</code> directly
Request attributes	<code>fail()</code> in <code>LoginServlet</code> , lines 126 to 129: <code>error</code> , <code>email</code> , <code>csrf</code> are read in the JSP as <code>\${error}</code>
HttpSession	Created <code>LoginServlet</code> line 79, user stored line 104, id changed line 98, read in <code>Web.current</code> line 51, destroyed <code>LogoutServlet</code> line 36, timeout <code>web.xml</code> line 23
Cookies	<code>LoginServlet.writeCookie</code> lines 171 to 180, <code>readCookie</code> lines 152 to 169, <code>ThemeServlet</code> lines 33 to 37, session cookie flags in <code>web.xml</code> lines 24 to 27
Filter	<code>AuthFilter</code> (<code>/app/*</code> , <code>/admin/*</code>) and <code>SecurityHeadersFilter</code> (<code>/*</code>), both calling <code>chain.doFilter</code>
Listener	<code>SessionTracker</code> : <code>@WebListener</code> , <code>sessionCreated</code> line 23, <code>sessionDestroyed</code> line 31. Read by <code>SystemService</code> lines 48 and 49
Error pages	<code>web.xml</code> lines 33 to 48: 404, 403, 500 and any uncaught <code>Throwable</code>
JSP, EL, JSTL	<code>login.jsp</code> uses <code>c:out</code> to escape every value and <code>c:if</code> for messages. Scriptlets are switched off in <code>web.xml</code> lines 58 and 67
EJB and injection	<code>AuthService</code> is <code>@Stateless</code> , injected with <code>@EJB</code> at <code>LoginServlet</code> line 39. Every call is logged by the audit interceptor with the password redacted

7. Questions you will be asked

Q How is an account created?

Student self sign up only, at `/register`. `RegisterServlet` checks the CSRF token, name, email (four checks), password strength, roll number, branch, batch year, CGPA, backlogs and phone. Only if there are no errors does it call `AuthService.registerStudent`, which checks for a duplicate email or roll number again, hashes the password, and saves an `app_user` row plus a student profile. The role is hard coded to STUDENT. Officers exist only through the seeder, never through sign up.

Q Would it accept a fake email?

It rejects an address whose domain does not exist: `test@thisdomaindoesnotexist123.com` gets "That domain doesn't appear to accept email". It rejects a malformed one such as `admin@campus`, and 43 known disposable providers such as mailinator. It would still accept an invented mailbox at a real domain, for example `asdf1234@gmail.com`, because we do not check the mailbox.

Q So do you verify the email really exists?

Done: shape check (`Validators.isEmail`), disposable blocklist, and a DNS lookup for an MX record with A and AAAA as fallback (`EmailDomainVerifier`). That proves the domain can receive mail. **Not done:** no confirmation link, no OTP, no test send, and nothing proves the person owns the mailbox. That needs an SMTP account we do not have. It is the recorded next step (Security review S10). If DNS cannot be reached at all, the check lets the sign up through and logs a warning, on purpose.

Q Does it hide the password?

Yes on screen: the inputs are `type="password"` on both forms. The forms use POST, so the password is in the body, not the URL. When a form fails, the servlet forwards the page back with no password value, so the boxes come back empty. The audit trail records the call as `authenticate(email, [redacted])`. Honest limit: the app runs on plain HTTP on localhost, so on a real network it would need HTTPS.

Q How is the password stored? Can you see it?

As `pbkdf2$120000$salt$hash` in `app_user.password_hash`: PBKDF2 with HMAC SHA 256, 120,000 rounds, a 16 byte random salt per account, 256 bit output (`PasswordHasher`). It is one way, so we cannot read the original. Login hashes the typed password with the stored salt and compares in constant time. The salt means two people with the same password get different hashes.

Q What happens after wrong passwords?

Each wrong password adds one to `failed_attempts` in the database (`AuthService` lines 74 to 85). The message is "Email or password is incorrect." with HTTP 401. With two or one attempts left it adds a countdown. The fifth sets `locked_until` fifteen minutes ahead. While locked, even the correct password is refused. A correct sign in resets the counter. The officer can clear a lockout early on the student's page. The lock lives in the database, so restarting the server does not clear it, and it counts per account, not per computer.

Q Can someone tell which emails have accounts?

Not from the wrong password message: unknown email and wrong password read the same, and an unknown email still burns one hash so the timing matches. Two small leaks exist and you should say so: the countdown and lockout messages only appear for real accounts, and the register page says "An account already exists for that address".

Q Can a student open the admin pages?

No. `AuthFilter` line 63 checks the role for every path under `/admin`, and a student gets HTTP 403 with the "Not allowed" page. Someone not signed in is redirected to login. The path it tests is cleaned first by `RequestPaths.canonical`, so `//admin/database` and `/admin;x=1/students` are refused too. We tested all of these as a student. This was a real bug in an earlier version: the check read the raw URL, the container routed on the cleaned one, and a doubled slash got through. It is fixed and covered by `RequestPathsTest`.

Q What is session fixation and how do you stop it?

An attacker plants a known session id in the victim's browser, waits for the victim to sign in, then uses the same id. We call `request.changeSessionId()` right after a successful password check (`LoginServlet` line 98), so the pre login id becomes useless. Only cookies are allowed for the id (`web.xml` line 30), so it cannot sit in a URL.

Q How long does a session last, and who counts them?

30 minutes of inactivity: `web.xml` line 23, and `SessionTracker` line 24 sets the same 30 minutes when each session is created. Logout invalidates it at once. `SessionTracker` implements `HttpSessionListener`; the container calls it on creation and on invalidation or timeout, and the officer's System page shows "Sessions open now".

Q Which cookies do you set, and with what flags?

`JSESSIONID`: `HttpOnly` (in `web.xml`). `remember_email` (only if the box is ticked) and `last_visit`: `HttpOnly`, 30 days, path `/placement`, values percent encoded. `portal_theme`: one year, not `HttpOnly`, only says light or dark. No cookie holds a password. Not set: the `Secure` flag, because a secure cookie would never be sent over plain HTTP on localhost (accepted risk A2 in the review), and no `SameSite` attribute is set in our code.

Q Why are the JSPs under WEB-INF? Forward or redirect?

Anything under `WEB-INF` cannot be requested by URL: we tested `/placement/WEB-INF/views/login.jsp` and got 404. Only a servlet can reach a view, by `RequestDispatcher.forward`, which happens inside the server: the address bar keeps the servlet URL. A redirect sends the browser a 302 and it makes a second request. We forward to show a page, including a failed login, and redirect after every successful POST (Post-Redirect-Get) so refresh cannot repeat the action.

Q What are CSRF and the Content Security Policy?

CSRF: another site making your browser submit a form to us. Every POST needs the random per session token in a hidden field (`Web.csrfValid`). **CSP:** `script-src 'self'` with no inline scripts, so the browser refuses any script we did not serve. Plus `frame-ancestors 'none'` and `X-Frame-Options: DENY` against clickjacking, `form-action 'self'` , `nosniff` , and no-store on every page that is not a static asset. Not set: HSTS, because we have no HTTPS.

8. If something goes wrong

Fixed: the admin role check bypass

What it was. The officer check tested the raw request URL with `startsWith("/admin/")` . The container routes on a cleaned path, so `/placement//admin/database` reached the officer Database page as a student. **Why.** The raw URL and the routed path are not the same text. **The fix.** `RequestPaths.canonical` (strip context path and `;` parameters, collapse repeated slashes) and `RequestPaths.isAdminPath` , used at `AuthFilter` lines 53 and 63. **Checked.** As a student, `/admin/database` , `//admin/database` , `//admin//students` and `/admin;x=1/students` all return 403. If asked, say plainly that we found it ourselves and closed it.

WHAT YOU SEE	WHY, AND WHAT TO DO
Sign up says the domain "doesn't appear to accept email" for a real address	DNS on this network answered that the name does not exist. Try another network, or use <code>gmail.com</code> . If DNS cannot be reached at all the code lets the sign up through, so a refusal means a definite "no".
"This page sat open too long to be verified" or "could not be verified"	The CSRF token did not match: the session expired (30 minutes), cookies are blocked, or the page was opened in another window. Reload the page and fill it in again.
"An account already exists" or "Roll number ... is already registered"	You already used that email or roll number in a rehearsal. Pick new ones, or remove the old test account as the officer.
Demo account is locked	Five wrong attempts. Wait 15 minutes, or as the officer open Students, open the account, click "Clear the logout".
Correct password refused with 401	Check whether the message says locked, deactivated, or "not a valid email address". Each has its own text. Otherwise check Caps Lock: the email is not case sensitive, the password is.
Signed out in the middle of the demo	30 minute inactivity timeout, or the server was restarted. Sign in again.
mysql says "Access denied ... using password: NO"	Set <code>\$env:MYSQL_PWD = 'PLacement@2026'</code> in the same PowerShell window first.
The app does not load	Do not restart it yourself mid presentation. Ask whoever started it to check the server window.

What is not implemented (say it plainly if asked)

- No email confirmation link, no OTP, no forgot password flow. A student who forgets a password cannot reset it by email. The change password action needs the current password (`AuthService.changePassword`).
- Only the email domain is checked. The mailbox is not.
- No HTTPS, so no `Secure` cookie flag and no HSTS. On a real network the password and session id would travel in clear text.
- Lockout is per account, not per address, and the seeded demo accounts share one password (accepted risk A6 in the security review).
- Logout works on a plain GET link, so another site could sign a user out. That is a nuisance, not a data risk.
- The roles are two only: officer and student. There is no role management screen.